Gestion de cinémas

«CINELY»

Rapport de Projet

Réalisé par :

KERAMANE Mazarine
MAOUCHE Lina
KHEREDDINE Amira
HEDJRESS Tesnim

Mai 2025

RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET POPULAIRE

Université Abderrahmane Mira Béjaïa

Département informatique

Faculté des sciences exactes

RAPPORT PROJET

THÈME :

« GESTION DE CINÉMAS »

Année scolaire 2024 / 2025

Table des matières

1 *Introduction* **2**
1.1 Contexte du projet : . . . 2
1.2 Objectifs . . . 2

2 *Analyse des besoins :* **4**
2.1 Besoins fonctionnels : . . . 4
2.2 Besoins non-fonctionnels : . . . 5

3 Conception : **6**
3.1 Technologies Utilisées : . . . 6
3.2 Outils de développement : . . . 6
3.3 Interface Utilisateur : . . . 6
3.3.1 Design : . . . 6
3.3.2 Navigation : . . . 7
3.3.3 Responsive : . . . 7
3.4 Sécurité et Performance : . . . 9
3.4.1 Sécurité : . . . 9
3.4.2 Performance : . . . 9
3.5 Tests et Validation : . . . 9
3.5.1 Tests Unitaires : . . . 9
3.5.2 Tests d'Intégration : . . . 9
3.5.3 Tests Utilisateurs : . . . 10
3.6 *Modélisation UML* . . . 11
3.6.1 Diagramme de classe . . . 11
3.6.2 Diagramme de Cas de Séquence . . . 12
3.6.3 Description textuelle du cas d'utilisation «Ajouter un Snack» . . . 13
3.6.4 Diagramme de Cas d'Utilisation . . . 14
3.7 *Modèle Rationnel* . . . 15
3.7.1 Modèle relationnel de l'application . . . 15
3.7.2 Modele Rationnel(schéma) . . . 17

4 *Annexe* **18**
4.1 *Script SQL de la création de la base de donnée :* . . . 18

5 *Réalisation* **20**

6 *Conclusion* **22**

1

Introduction

1.1 Contexte du projet :

En Algérie, le cinéma connaît un regain d'intérêt, mais les méthodes traditionnelles de réservation de places présentent des défis importants. Les cinémas font face à des problèmes de gestion manuelle des réservations, d'expérience utilisateur limitée et d'absence de services additionnels. Notre application vise à moderniser ce processus en offrant une solution numérique complète.
Problématique : Comment développer une application qui optimise le processus de réservation de places de cinéma en Algérie tout en améliorant l'expérience utilisateur ?

Cette problématique se décline en plusieurs sous-questions :

Technique : Comment concevoir une application robuste et sécurisée qui gère efficacement les réservations en temps réel ?
Utilisateur : Comment offrir une expérience utilisateur intuitive et agréable qui simplifie le processus de réservation ?
Fonctionnelle : Comment intégrer des services additionnels (comme la vente de snacks) tout en maintenant la simplicité d'utilisation ?
Sécurité : Comment garantir la sécurité des données et des transactions tout en assurant une expérience fluide ?
Évolutivité : Comment concevoir une solution qui puisse évoluer et s'adapter aux besoins futurs du marché ?

1.2 Objectifs

L'objectif de ce projet est double : d'une part, développer une application de gestion complète pour une chaîne de cinéma , et d'autre part, renforcer nos compétences pratiques en tant qu'étudiants à travers la réalisation d'un projet concret L'application vise à centraliser et automatiser la gestion des cinémas, des salles, des films et des projections afin de :

— Simplifier les tâches administratives grâce à une interface facile d'utilisation et efficace. exp(le processus de réservation)

— Assurer la fiabilité et la traçabilité des données via une base de données relationnelle robuste.
— Digitaliser la gestion des cinemas et surtout en Algerie
— Offrir des fonctionnalités de recherche et de gestion avancées, adaptées aux besoins des utilisateurs et afin d'améliorer l'expérience utilisateur.

Notre application permettra donc de :
— l'Authentification des utilisateurs
— Consultation des films
— Réservation de places
— Séléction de snacks
— paiement et confirmation
— Génération de ticket

Et en tant qu'étudiants, ce projet nous permettra de :
— Mettre en pratique les concepts appris , notamment en modélisation UML, conception de bases de données, et développement d'interfaces.
— Collaborer efficacement en groupe pour concevoir et implémenter une solution répondant aux exigences.
— Acquérir une expérience enrichissante dans le cycle de vie d'un projet logiciel, depuis l'analyse des besoins jusqu'à la démonstration finale.

2

Analyse des besoins :

2.1 Besoins fonctionnels :

Voici les différentes Fonctionnalités implémentées lors du développement de notre application :

1. Authentification des utilisateurs : L'application permet aux utilisateurs de s'inscrire et de se connecter de manière sécurisée. Une fois connectés, les utilisateurs restent authentifiés grâce à la gestion de session.

2. Consultation des films : Les utilisateurs peuvent consulter la liste des films disponibles à l'affiche. Chaque film affiche des détails comme le titre, le synopsis, la durée et l'affiche du film.

3. Réservation de places : Lorsqu'un utilisateur sélectionne un film, il peut réserver une ou plusieurs places. Le plan de salle affiche clairement les places disponibles, réservées et sélectionnées, facilitant ainsi le choix.

4. Sélection de snacks : En plus des places, les utilisateurs peuvent ajouter des snacks à leur réservation. La liste des snacks disponibles est présentée, et le montant total est automatiquement mis à jour.

5. Paiement et confirmation : Le système calcule automatiquement le montant total de la réservation (places et snacks). L'utilisateur peut alors valider et confirmer sa réservation.

6. Génération de ticket : Après le paiement, un ticket électronique est généré. Ce ticket inclut un QR code contenant les informations de la réservation et affiche les détails comme le film, la place, les snacks, le montant et la date.

7. Gestion de l'état et navigation : L'application utilise Provider pour gérer l'état global, assurant une navigation fluide entre les différentes pages (authentification, films, réservation, ticket, etc.).

8. Expérience utilisateur : L'interface est moderne, responsive et intuitive. Les places disponibles, réservées et sélectionnées sont clairement indiquées par des codes couleur, et des messages d'erreur ou de confirmation sont affichés pour guider l'utilisateur.

9. Administration : Des scripts permettent l'initialisation automatique des places pour chaque film dans la base de données, ainsi que l'importation de films et la gestion des données.

2.2 Besoins non-fonctionnels :

1.Sécurité : Protection des données utilisateurs et des transactions.

2.Performance : Temps de réponse rapide pour les requêtes API.

3.Interface Utilisateur : Design moderne et intuitif pour une expérience utilisateur optimale.

4.Compatibilité : Fonctionnement sur différentes plateformes (desktop, mobile).

5.Évolutivité : Architecture permettant l'ajout de nouvelles fonctionnalités.

3

Conception :

3.1 Technologies Utilisées :

1.Frontend :

- **Flutter :** Framework de développement multiplateforme pour créer l'interface utilisateur de l'application.
- **Provider :** Bibliothèque de gestion d'état pour gérer l'état global de l'application.
- **Shared Preferences :** Bibliothèque pour stocker des données locales sur l'appareil de l'utilisateur.
- **QR Flutter :** Bibliothèque pour générer des QR codes dans l'application.

2.Backend :

- **PHP :** Langage de programmation utilisé pour développer l'API backend.
- **MySQL :** Système de gestion de base de données pour stocker les informations des utilisateurs, des films, des places, des snacks et des réservations.
- **WAMP :** Environnement de développement local pour exécuter le serveur web, PHP et MySQL.

3.2 Outils de développement :

- **Git :** Système de contrôle de version pour gérer le code source.
- **Postman :** Outil pour tester les API et vérifier les requêtes HTTP.
- **phpMyAdmin :** Interface web pour gérer la base de données MySQL.

3.3 Interface Utilisateur :

3.3.1 Design :

-Flutter : Framework utilisé pour développer l'interface utilisateur, offrant un design moderne et intuitif.
-Material Design : Utilisation des composants Material Design de Flutter pour garantir une interface cohérente et esthétique.
-Couleurs et Typographie : Palette de couleurs et polices choisies pour améliorer la lisibilité et l'attrait visuel.

3.3.2 Navigation :

-Provider : Utilisé pour la gestion de l'état global de l'application, facilitant la navigation entre les différentes pages.
-Routes : Définition des routes pour naviguer entre les écrans (authentification, films, réservation, ticket, etc.).
-Navigation Flutter : Utilisation de Navigator pour gérer la pile de navigation et les transitions entre les écrans.

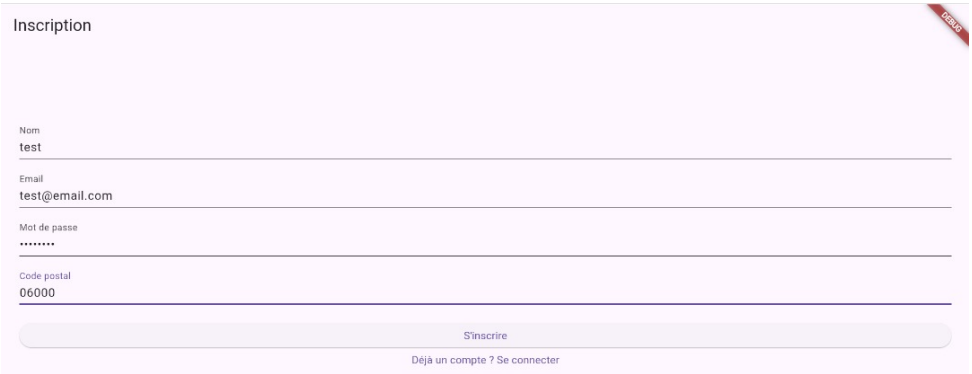
3.3.3 Responsive :

-Layouts Adaptatifs : Utilisation de LayoutBuilder et MediaQuery pour adapter l'interface aux différentes tailles d'écran.
-Flexible et Expanded : Widgets Flutter utilisés pour créer des layouts flexibles qui s'adaptent à l'espace disponible.

Aperçu de l'interface utilisateur à travers quelques illustrations :

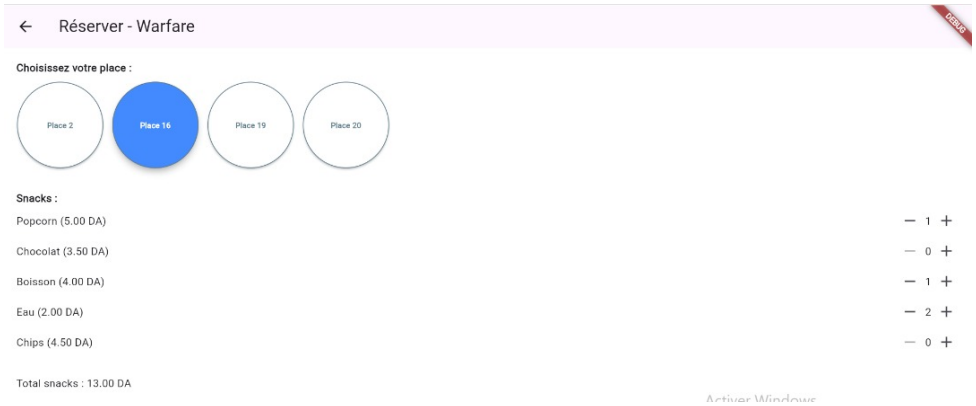
1ère page :

2ème page :



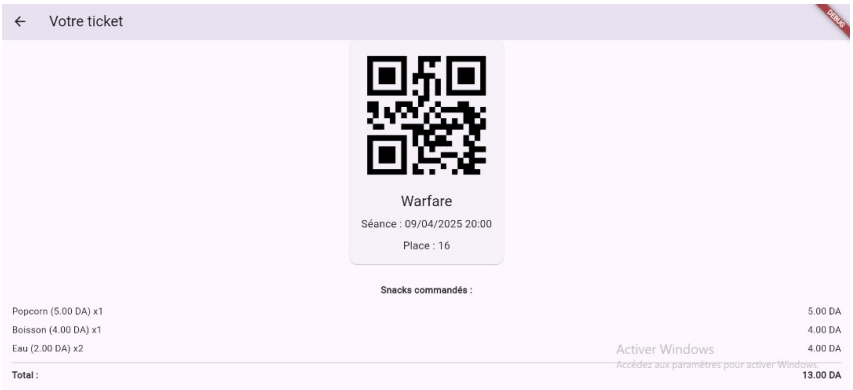
3ème page :

4ème page :



5ème page :

6ème page :



3.4 Sécurité et Performance :

3.4.1 Sécurité :

— **Chiffrement des Mots de Passe :** Utilisation de fonctions de hachage pour sécuriser les mots de passe des utilisateurs.

— **Validation des Données :** Vérification des entrées utilisateur côté serveur pour prévenir les injections SQL et autres attaques.

— **CORS :** Configuration des en-têtes CORS pour sécuriser les requêtes API.

3.4.2 Performance :

— **Requêtes HTTP Optimisées :** Utilisation de la bibliothèque http pour effectuer des requêtes API efficaces.

— **Mise en Cache :** Stockage local des données avec SharedPreferences pour réduire les requêtes réseau.

— **Lazy Loading :** Chargement des images et des données uniquement lorsque nécessaire pour améliorer les performances.

3.5 Tests et Validation :

3.5.1 Tests Unitaires :

— **Flutter Test :** Framework de test utilisé pour écrire des tests unitaires pour les composants individuels de l'application.

— **Mockito :** Bibliothèque utilisée pour simuler les dépendances lors des tests unitaires.

3.5.2 Tests d'Intégration :

— **Flutter Integration Test :** Tests d'intégration pour vérifier les interactions entre les composants de l'application.

— **API Testing :** Tests des endpoints API pour s'assurer qu'ils fonctionnent correctement.

3.5.3 Tests Utilisateurs :

— **Retours Utilisateurs :** Collecte de retours des utilisateurs pour identifier les problèmes et améliorer l'application.

— **Beta Testing :** Phase de test avec un groupe d'utilisateurs pour valider l'application avant le déploiement.

3.6 *Modélisation UML*

3.6.1 Diagramme de classe

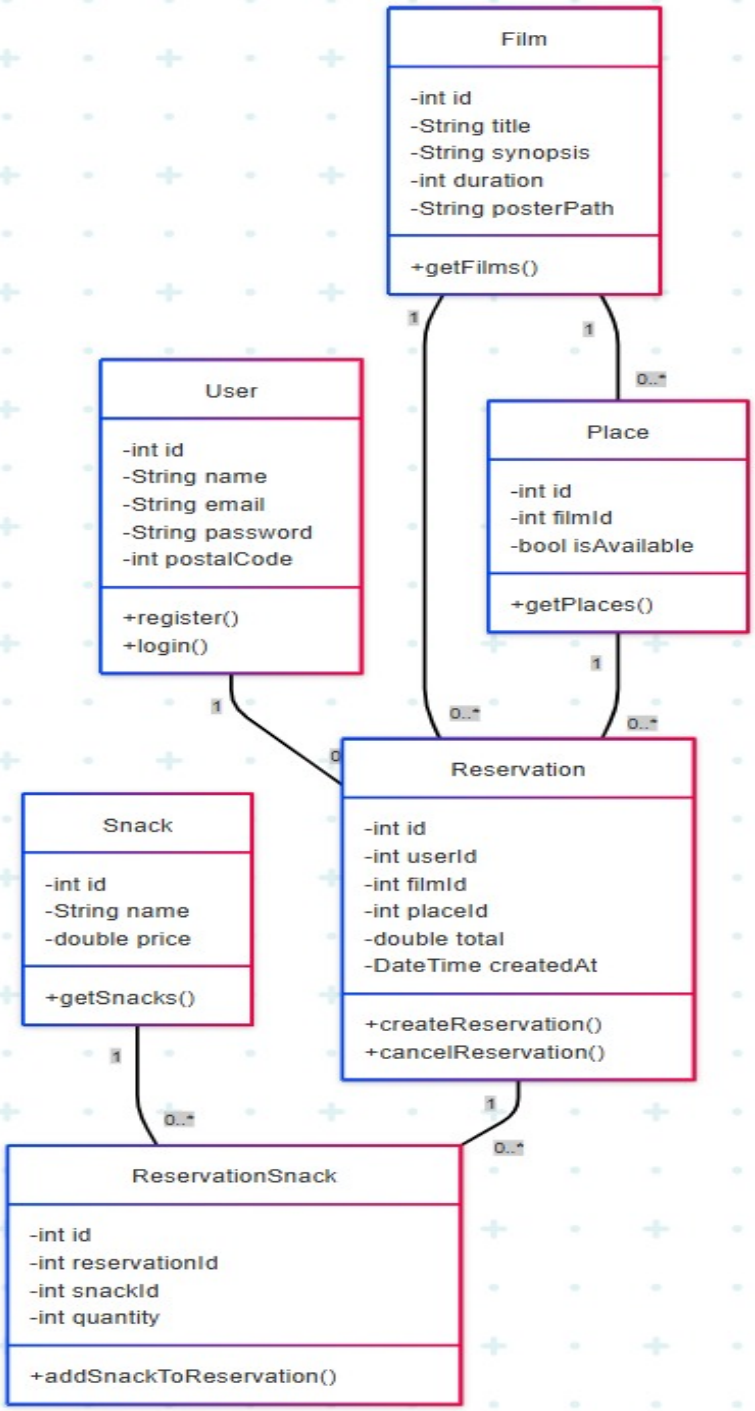


FIGURE 3.1 – Diagramme de classe

3.6.2 Diagramme de Cas de Séquence

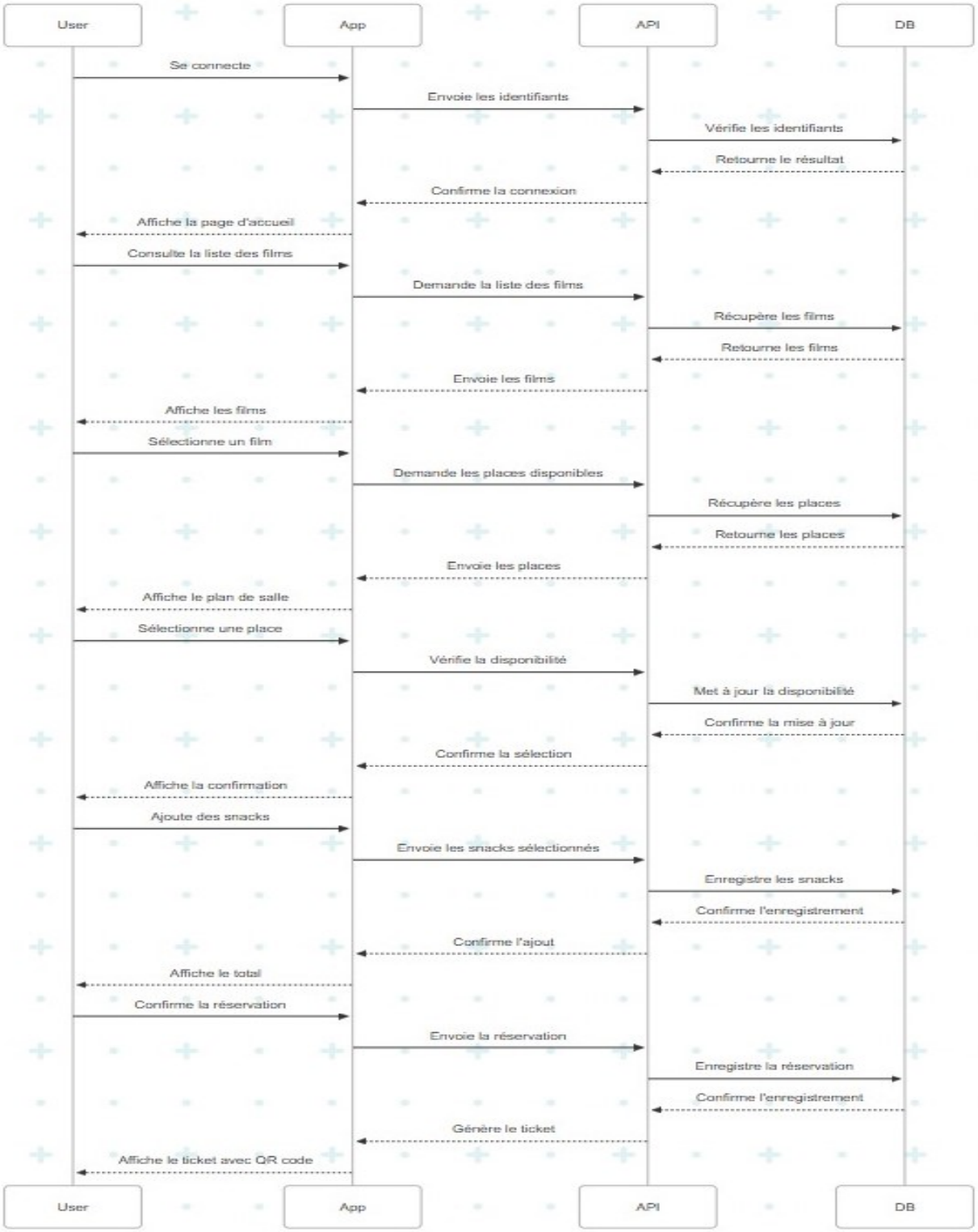


FIGURE 3.2 – Interractions de l'applications "

3.6.3 Description textuelle du cas d'utilisation «Ajouter un Snack»

Titre	Ajouter un snack
Acteur principal	Administrateur du cinéma
Objectif principal	Ajouter un nouveau snack à la liste des snacks proposés lors de la réservation. |
Préconisions	L'administrateur est connecté et a accès à la gestion des snacks.
Post-conditions	Le snack est enregistré dans le système avec un identifiant unique et devient disponible pour les utilisateurs
Flux principal	1. L'administrateur accède à la page de gestion des snacks. 2. Il clique sur « Ajouter un snack ». 3. Le système affiche un formulaire d'ajout. 4. L'administrateur saisit le nom, le prix et la description du snack. 5. Il clique sur « Valider ». 6. Le système vérifie les informations. 7. Le système enregistre le snack et affiche un message de confirmation. |
Scénarios alternatifs	5a. L'administrateur clique sur « Annuler » :;Le système annule l'opération et retourne à la liste des snacks.6a. Les informations sont incomplètes ou incorrectes :;Le système affiche un message d'erreur et demande de corriger les champs.
Exception	- Le nom du snack existe déjà : le système affiche un message d'erreur

FIGURE 3.3 – cas d'utilisation «Ajouter un Snack»

3.6.4 Diagramme de Cas d'Utilisation

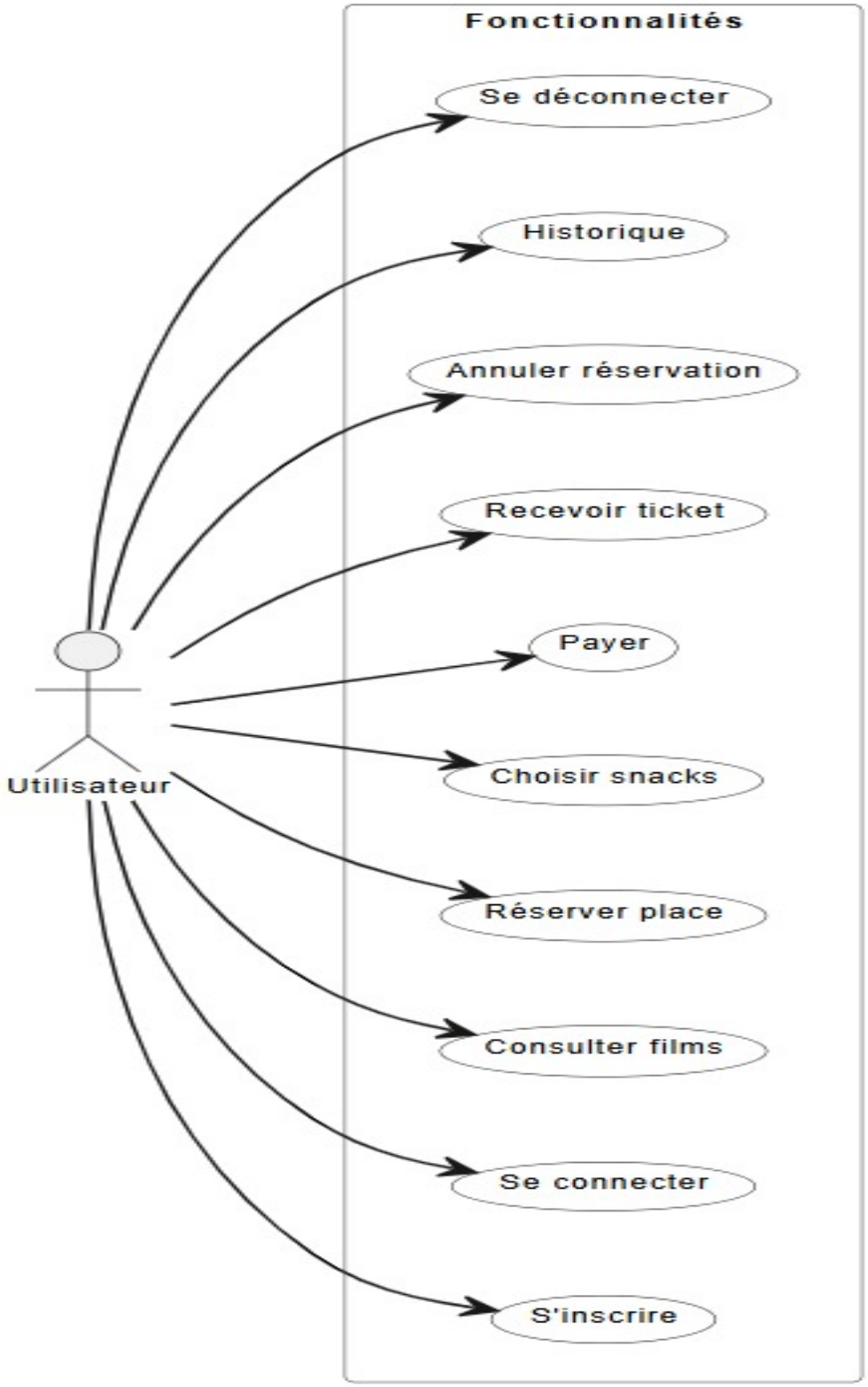


FIGURE 3.4 – Cas d'utilisation de l'application

3.7 *Modèle Rationnel*

3.7.1 Modèle relationnel de l'application

```
1. Table users
*id (INT, PRIMARY KEY, AUTO_INCREMENT) : Identifiant unique
de l'utilisateur.
*name (VARCHAR) : Nom de l'utilisateur.
*email (VARCHAR) : Adresse email de l'utilisateur (unique).
*password (VARCHAR) : Mot de passe de l'utilisateur (hashe).
*postal_code (INT) : Code postal de l'utilisateur.


2. Table films
*id (INT, PRIMARY KEY, AUTO_INCREMENT) : Identifiant unique du film.
*title (VARCHAR) : Titre du film.
*synopsis (TEXT) : Synopsis du film.
*duration (INT) : Dur e du film en minutes.
*poster_path (VARCHAR) : Chemin de l'affiche du film.


3. Table places
*id (INT, PRIMARY KEY, AUTO_INCREMENT) : Identifiant unique de la place.
*film_id (INT, FOREIGN KEY) : Identifiant du film associer    la place.
*is_available (BOOLEAN) : Indique si la place est disponible ou non.


4. Table snacks
*id (INT, PRIMARY KEY, AUTO_INCREMENT) : Identifiant unique du snack.
*name (VARCHAR) : Nom du snack.
*price (DECIMAL) : Prix du snack.


5. Table reservations
*id (INT, PRIMARY KEY, AUTO_INCREMENT): Identifiant unique
de la reservation.
*user_id (INT, FOREIGN KEY): Identifiant de l'utilisateur qui
a fait la reservation.
*film_id (INT, FOREIGN KEY): Identifiant du film reserver.
*place_id (INT, FOREIGN KEY): Identifiant de la place reserver.
*total (DECIMAL): Montant total de la reservation.
*created_at (DATETIME) : Date et heure de creation de la reservation.
```

6. Table reservation_snack
*id (INT, PRIMARY KEY, AUTO_INCREMENT): Identifiant unique
de l'association.
*reservation_id (INT, FOREIGN KEY): Identifiant de la reservation.
*snack_id (INT, FOREIGN KEY): Identifiant du snack associ
la reservation.
*quantity (INT): Quantit de snacks associer a la reservation.

3.7.2 Modele Rationnel(schéma)

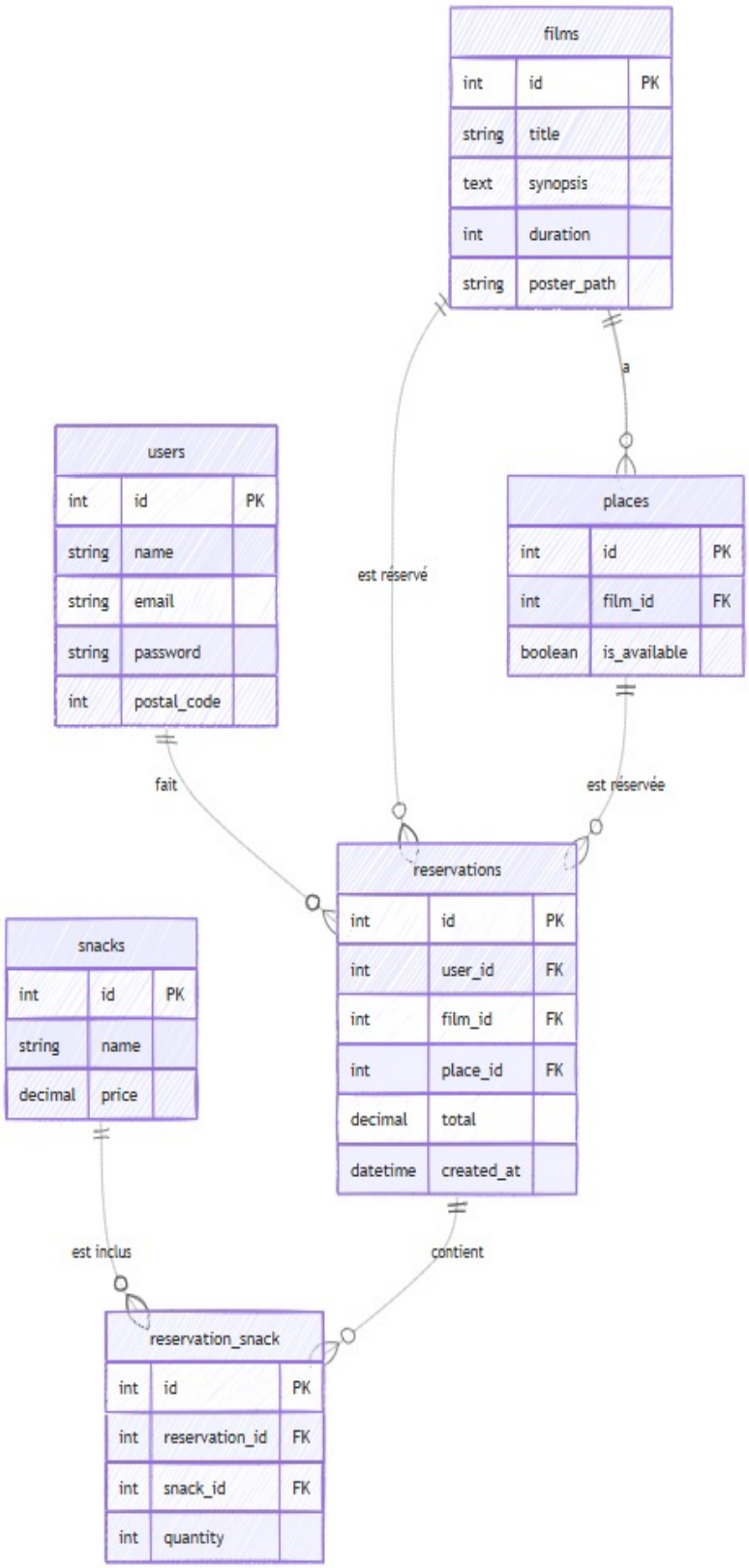


FIGURE 3.5 – Schématisation du modèle rationnel

4

Annexe

4.1 *Script SQL de la création de la base de donnée :*

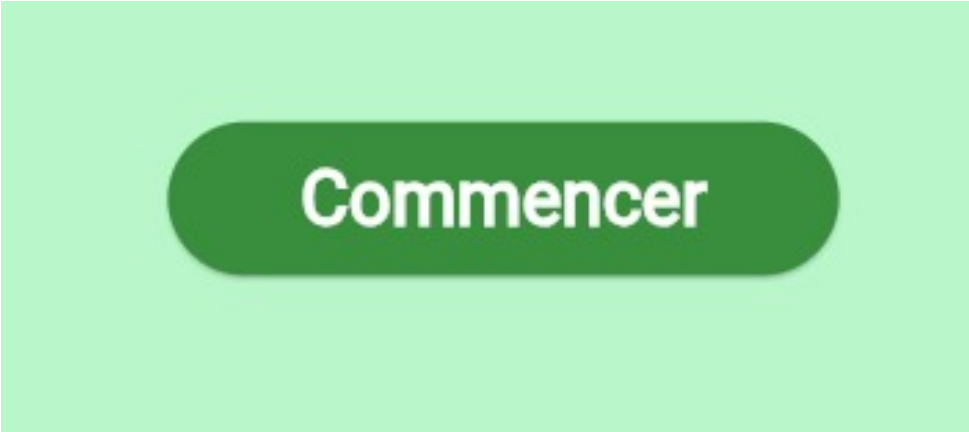
```sql
-- Table des utilisateurs
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    mot_de_passe VARCHAR(255) NOT NULL,
    code_postal INT NOT NULL
);



-- Table des films
CREATE TABLE films (
    id INT AUTO_INCREMENT PRIMARY KEY,
    titre VARCHAR(255) NOT NULL,
    description TEXT,
    miniature VARCHAR(255),
    date_heure DATETIME NOT NULL
);



-- Table des places
CREATE TABLE places (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_film INT NOT NULL,
    numero_place INT NOT NULL,
    est_occupee TINYINT(1) DEFAULT 0,
    FOREIGN KEY (id_film) REFERENCES films(id)
    ON DELETE CASCADE
);
```

```sql
-- Table des snacks
CREATE TABLE snacks (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    prix DECIMAL(6,2) NOT NULL
);



-- Table des reservations
CREATE TABLE reservations (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_user INT NOT NULL,
    id_film INT NOT NULL,
    id_place INT NOT NULL,
    total DECIMAL(8,2) NOT NULL,
    date_reservation DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_user) REFERENCES users(id),
    FOREIGN KEY (id_film) REFERENCES films(id),
    FOREIGN KEY (id_place) REFERENCES places(id)
);



-- Table de liaison reservations/snacks
CREATE TABLE reservation_snack (
    id INT AUTO_INCREMENT PRIMARY KEY,
    id_reservation INT NOT NULL,
    id_snack INT NOT NULL,
    quantite INT NOT NULL,
    FOREIGN KEY (id_reservation) REFERENCES reservations(id)
    ON DELETE CASCADE,
    FOREIGN KEY (id_snack) REFERENCES snacks(id)
);
```

5

Réalisation

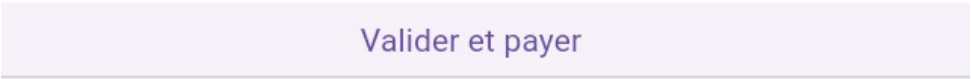
Ce chapitre présente l'implémentation concrète de l'application à travers des captures d'écran illustrant ses différentes interfaces et fonctionnalités (Boutons).
1ère image :Écran d'accueil (Welcome Screen) Bouton "Commencer" : Un bouton vert qui permet aux utilisateurs de commencer à utiliser l'application et les redirige vers l'écran de connexion



2ème image :Écran de connexion (Login Screen) Bouton "Se connecter" : Permet aux utilisateurs de se connecter avec leur email et mot de passe. Lien "Pas de compte ? S'inscrire" : Redirige les utilisateurs vers l'écran d'inscription s'ils n'ont pas encore de compte.

3ème image :Écran d'accueil principal (Home Screen) Bouton de déconnexion (icône de déconnexion) : Situé dans la barre d'application, permet aux utilisateurs de se déconnecter. Bouton "Réserver" : Apparaît pour chaque film dans la liste, permet de réserver une séance pour le film sélectionné.

4ème image :Écran de réservation (Reservation Screen) Boutons de sélection de place : Des boutons circulaires représentant les places disponibles dans la salle. Bouton "Valider et payer" : Permet de finaliser la réservation après avoir sélectionné une place.



5ème page :Boutons de sélection de place : Des boutons circulaires représentant les places disponibles dans la salle.

6

Conclusion

Ce projet pluridisciplinaire nous a permis de mettre à profit nos compétences techniques, organisationnelles et créatives pour développer une solution complète de gestion de cinémas. Grâce à l'application "CINELY", nous avons répondu à plusieurs besoins spécifiques, notamment la centralisation des informations, les réservations en ligne et la communication client.

Nous avons intégré des concepts clés en matière de développement Desktop, d'administration de bases de données, de sécurité informatique et de gestion de projet, tout en conservant une approche centrée sur l'utilisateur.

Ce travail nous a également permis de mieux comprendre les implications culturelles de la transformation numérique et de renforcer notre capacité à travailler en équipe.